OpenClaw 快速上手指南

精简版 · 10 章掌握核心概念与关键配置

AI 智能体 Gateway 网关

OpenClaw 项目组

2026 年 3 月 11 日

版权声明

本书内容基于 OpenClaw 项目官方文档整理编写。

OpenClaw 项目采用 MIT 许可证，版权归 Peter Steinberger 及其贡献者所有。

原始项目：https://github.com/anthropics/openclaw
原始许可证：MIT License
Copyright © 2025 Peter Steinberger

本书同样采用 **MIT 许可证**进行授权。

MIT License

Copyright © 2025 OpenClaw 项目组

Permission is hereby granted, free of charge, to any person obtaining a copy of this book and associated documentation files (the "Book"), to deal in the Book without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Book, and to permit persons to whom the Book is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Book.

THE BOOK IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.

目录

1 快速上手 **8**

1.1 OpenClaw 简介 8

1.1.1 工作原理 8

1.2 三步安装 8

1.2.1 第一步：安装 OpenClaw 8

1.2.2 第二步：运行新手引导 8

1.2.3 第三步：启动 Gateway 9

1.3 最快体验：Control UI 9

1.4 认证配置 9

1.4.1 Anthropic API 密钥（推荐） 9

1.4.2 验证安装 9

1.5 核心概念速览 10

1.6 配置文件 10

2 Gateway（网关核心） **11**

2.1 概述 11

2.2 启动方式 11

2.2.1 前台运行 11

2.2.2 守护进程模式 11

2.3 智能体循环（Agent Loop） 12

2.4 RPC 接口 12

2.5 认证 12

2.6 常用运维命令 13

3 Agent（智能体） **14**

3.1 概述 14

3.2 单智能体模式（默认） 14

3.3 多智能体模式 14

3.3.1 添加智能体 15

3.3.2 配置示例 15

3.4 工作区文件 15

3.5 智能体超时 16

3.6 常用命令 16

4 Channel（渠道） 17
4.1 概述 17
4.2 内置渠道 17
4.3 渠道登录 17
4.4 访问控制 18
4.4.1 私信策略（dmPolicy） 18
4.4.2 群组策略（groupPolicy） 18
4.4.3 配置示例 18
4.5 路由机制 19
4.5.1 会话键格式 19
4.6 多账户 19

5 Binding（路由绑定） 20
5.1 概述 20
5.2 路由优先级 20
5.3 核心概念 20
5.4 配置示例 21
5.4.1 两个 WhatsApp 账户 → 两个 Agent 21
5.4.2 一个号码按发送者分流 21
5.4.3 特定群组路由到特定 Agent 22
5.5 验证 22

6 Session（会话） 23
6.1 概述 23
6.2 会话所有权 23
6.2.1 存储位置 23
6.3 会话键映射 23
6.4 私信分组策略（dmScope） 23
6.5 会话生命周期 24
6.5.1 重置策略 24
6.5.2 手动重置 24
6.6 配置示例 24
6.7 常用命令 25

7 Plugin（插件） 26
7.1 概述 26
7.2 安装插件 26
7.3 插件清单 26

7.4 注册智能体工具 27
7.4.1 可选工具 27
7.5 插件钩子 28
7.6 配置示例 28
7.7 社区插件 28

8 Skill（技能） 30
8.1 概述 30
8.2 加载位置与优先级 30
8.3 SKILL.md 格式 30
8.3.1 Frontmatter 关键字段 31
8.4 门控（加载时过滤） 31
8.5 配置覆盖 31
8.6 ClawHub（Skills 注册表） 32
8.7 安全注意事项 32

9 Memory（记忆） 33
9.1 概述 33
9.2 记忆文件 33
9.3 何时写入记忆 33
9.4 自动记忆刷新 33
9.5 向量记忆搜索 34
9.5.1 默认行为 34
9.5.2 配置提供商 34
9.5.3 额外记忆路径 35
9.6 全文搜索 35
9.7 记忆搜索工具 35

10 Sandbox（沙箱隔离） 36
10.1 概述 36
10.2 什么会被沙箱隔离 36
10.3 沙箱模式 36
10.4 作用域 36
10.5 工作区访问 37
10.6 最小启用示例 37
10.7 镜像构建 37
10.8 自定义绑定挂载 38
10.9 setupCommand 38

10.10 多智能体覆盖 39
10.11 调试 39

11 实战：服务器部署与配置详解 40
11.1 远程服务器部署 40
11.1.1 环境信息 40
11.1.2 SSH 连接 40
11.1.3 安装 OpenClaw 40
11.2 API 配置 42
11.2.1 配置方式 42
11.2.2 配置文件结构 43
11.2.3 关键字段说明 43
11.2.4 常见配置错误 43
11.2.5 启动网关 43
11.2.6 验证配置 44
11.3 `~/.openclaw/` 目录结构详解 44
11.3.1 根级文件详解 46
11.3.2 `agents/` — Agent 注册与会话存储 47
11.3.3 `canvas/` — Canvas 画布 48
11.3.4 `completions/` — Shell 自动补全 49
11.3.5 `cron/` — 定时任务 49
11.3.6 `devices/` — 设备管理 49
11.3.7 `identity/` — 身份认证 50
11.3.8 `logs/` — 审计日志 50
11.3.9 `memory/` — 记忆数据库 50
11.3.10 `workspace/` — 工作空间（人设与知识库） 51
11.4 Skills 系统详解 55
11.4.1 Skills 三层存放路径 55
11.4.2 Skill 的目录结构 55
11.4.3 Skill 的依赖检测与激活 56
11.4.4 Skills 管理命令 57
11.4.5 安装第三方 Skills 57
11.4.6 内置 Skills 一览（51 个） 59
11.5 Agent 与会话层级关系 59
11.5.1 层级结构 59
11.5.2 Agent 管理 60
11.5.3 Session（会话）管理 60
11.5.4 Agent 独立人设配置 61

11.6 模型切换实战 62
11.6.1 切换步骤 62
11.6.2 关键注意事项 64
11.6.3 快速回退 65
11.7 实战踩坑总结 65

Chapter 1

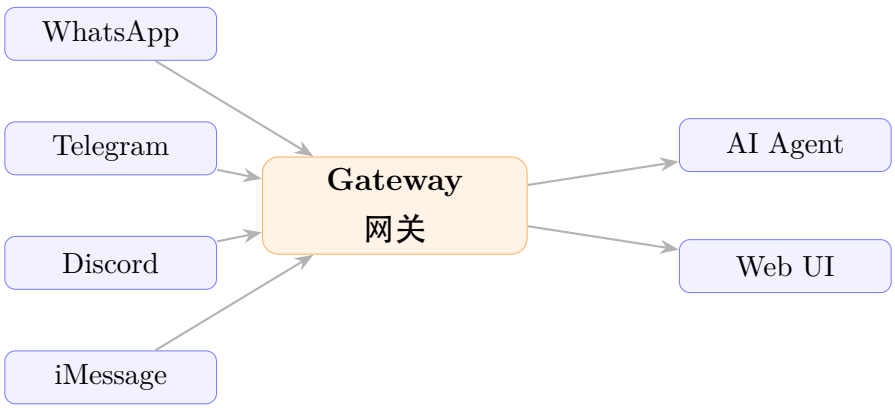
快速上手

1.1 OpenClaw 简介

”去壳！去壳！”—大概是一只太空龙虾说的

OpenClaw 是一个 AI 智能体 Gateway 网关，支持 WhatsApp、Telegram、Discord、iMessage 等多种聊天平台。通过单个 Gateway 网关进程将聊天应用连接到 AI 编程智能体，支持本地或远程部署。

1.1.1 工作原理



Gateway 网关是会话、路由和渠道连接的**唯一事实来源**。

1.2 三步安装

1.2.1 第一步：安装 OpenClaw

```
npm install -g openclaw@latest
```

1.2.2 第二步：运行新手引导

```
openclaw onboard --install-daemon
```

新手引导向导会自动配置：

- 模型/认证（推荐 OAuth 或 API 密钥）
- 渠道登录（WhatsApp、Telegram 等）
- 守护进程（systemd/launchd 开机自启）

1.2.3 第三步：启动 Gateway

```
openclaw channels login
openclaw gateway --port 18789
```

打开浏览器访问 `http://127.0.0.1:18789/` 即可开始聊天。

1.3 最快体验：Control UI

无需任何渠道配置，直接用浏览器聊天：

```
openclaw dashboard
```

在浏览器中打开 `http://127.0.0.1:18789/`，即可与智能体对话。

1.4 认证配置

1.4.1 Anthropic API 密钥（推荐）

```
export ANTHROPIC_API_KEY="..."
openclaw models status
```

或将密钥写入守护进程配置：

```
cat >> ~/.openclaw/.env <<'EOF'
ANTHROPIC_API_KEY=...
EOF
```

1.4.2 验证安装

```
openclaw models status
openclaw doctor
```

1.5 核心概念速览

OpenClaw 由以下核心组件构成，后续章节将逐一详解：

概念	说明	配置路径
Gateway	AI 网关核心，WS/HTTP 服务器	`gateway.*`
Agent	AI 助手实例，绑定模型/技能	`agents.list[]`
Channel	聊天平台适配器	`channels.<name>.*`
Binding	用户/群组到 Agent 的路由	`agents.list[].bindings[]`
Session	一次会话，含上下文历史	`session.*`
Plugin	扩展模块，注册工具/渠道/Hook	`plugins.entries.*`
Skill	Agent 技能（工具 + 系统提示）	`skills.entries.*`
Memory	长期记忆，向量 + 全文搜索	`memorySearch.*`
Sandbox	Docker 沙箱隔离执行	`agents.defaults.sandbox.*`

1.6 配置文件

配置文件位于 `~/.openclaw/openclaw.json`（JSON5 格式）。

```
{
  channels: {
    whatsapp: {
      allowFrom: ["+15555550123"],
      groups: { "*": { requireMention: true } }
    }
  },
  messages: {
    groupChat: { mentionPatterns: ["@openclaw"] }
  }
}
```

如果你**不做任何修改**，OpenClaw 将使用内置的 Pi 二进制文件以 RPC 模式运行，并按发送者创建独立会话。

Chapter 2

Gateway（网关核心）

2.1 概述

Gateway 网关是 OpenClaw 的核心进程，承担以下职责：

- **消息路由**：接收各渠道入站消息，路由到正确的 Agent
- **会话管理**：维护所有会话状态（唯一事实来源）
- **智能体运行时**：内嵌 pi-agent-core，执行智能体循环
- **RPC 服务**：对外提供 WebSocket/HTTP API
- **插件宿主**：加载并运行所有已注册插件

配置路径：`gateway.*`

2.2 启动方式

2.2.1 前台运行

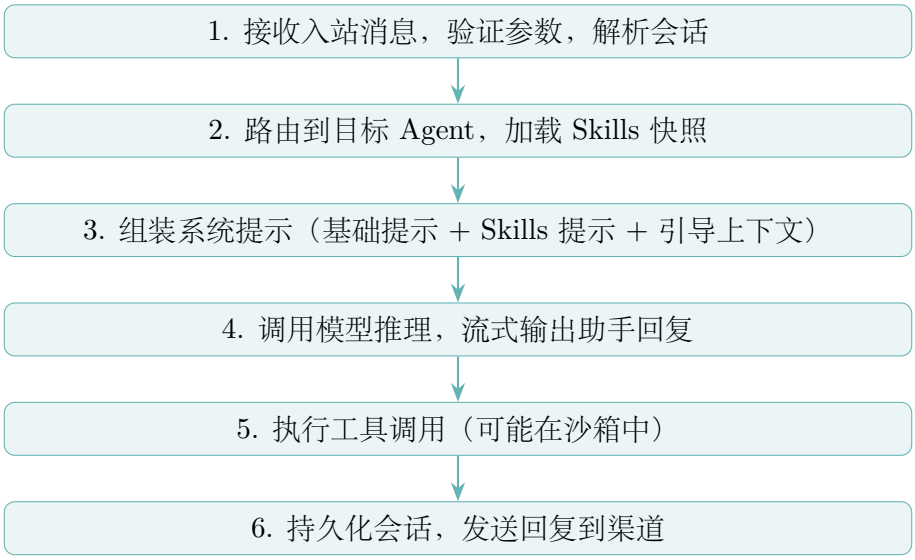
```
openclaw gateway --port 18789
```

2.2.2 守护进程模式

```
openclaw daemon install   # Install systemd/launchd service
openclaw daemon start     # Start the daemon
openclaw daemon status    # Check status
```

2.3 智能体循环（Agent Loop）

每条入站消息触发一次完整的智能体循环：



2.4 RPC 接口

Gateway 网关通过 WebSocket 和 HTTP 暴露 RPC 接口：

- `agent` / `agent.wait`：触发智能体运行
- `sessions.list`：列出会话
- `voicecall.*`：语音通话（插件）
- `message.send`：发送消息到渠道

示例（通过 CLI 调用 RPC）：

```
openclaw gateway call sessions.list --params '{}'
```

2.5 认证

推荐使用 **Anthropic API 密钥**：

```
export ANTHROPIC_API_KEY="..."
# Or store in daemon env file:
echo 'ANTHROPIC_API_KEY=...' >> ~/.openclaw/.env
```

也支持 OAuth 和 setup-token 流程。使用 `openclaw models status` 验证认证状态。

2.6 常用运维命令

```
openclaw doctor            # Health check
openclaw status            # Show storage paths & recent sessions
openclaw models status     # Check model auth
openclaw gateway --port N  # Start with custom port
```

Chapter 3

Agent（智能体）

3.1 概述

Agent（智能体）是 OpenClaw 中一个完全独立作用域的”大脑”，拥有自己的：

- **工作区**（文件、AGENTS.md/SOUL.md/USER.md、本地笔记、人设规则）
- **状态目录**（`agentDir`）：认证配置文件、模型注册表
- **会话存储**（聊天历史 + 路由状态）：`~/.openclaw/agents/<id>/sessions`

配置路径：`agents.list[]`

3.2 单智能体模式（默认）

不做任何配置时，OpenClaw 运行单个智能体：

- `agentId` 默认为 `main`
- 工作区默认为 `~/.openclaw/workspace`
- 状态默认为 `~/.openclaw/agents/main/agent`

3.3 多智能体模式

Gateway 网关可托管多个智能体并行运行，每个智能体拥有：

- 不同的工作区和人格（AGENTS.md、SOUL.md）
- 独立的认证和会话
- 独立的 Skills

3.3.1 添加智能体

```
openclaw agents add work
openclaw agents list --bindings
```

3.3.2 配置示例

```
{
  agents: {
    list: [
      {
        id: "home",
        default: true,
        name: "Home",
        workspace: "~/.openclaw/workspace-home"
      },
      {
        id: "work",
        name: "Work",
        workspace: "~/.openclaw/workspace-work"
      }
    ]
  }
}
```

3.4 工作区文件

工作区是智能体的家目录，包含以下标准文件：

文件	用途
AGENTS.md	操作指南（每个会话开始时加载）
SOUL.md	人设、语气和边界
USER.md	用户偏好
memory/	记忆文件目录
skills/	每智能体 Skills

3.5 智能体超时

- 默认超时：`agents.defaults.timeoutSeconds`（默认 600 秒）
- `agent.wait` 超时：默认 30 秒（仅等待，不停止智能体）

3.6 常用命令

```
# Run agent directly (no inbound chat)
openclaw agent --to +15555550123 --message "status update"
openclaw agent --agent ops --message "Summarize logs"

# List agents
openclaw agents list --bindings
```

Chapter 4

Channel（渠道）

4.1 概述

Channel（渠道）是 OpenClaw 与聊天平台之间的适配器。每个渠道负责：

- 接收平台入站消息并转换为统一格式
- 将智能体回复发送回对应平台
- 处理平台特有功能（附件、回应、线程等）

配置路径：`channels.<name>.*`

4.2 内置渠道

渠道 ID	平台	登录方式
`whatsapp`	WhatsApp	QR 码配对
`telegram`	Telegram	Bot Token
`discord`	Discord	Bot Token
`slack`	Slack	OAuth/Token
`signal`	Signal	配对
`imessage`	iMessage（旧版）	macOS
`bluebubbles`	iMessage（推荐）	BlueBubbles 服务器
`webchat`	Web 聊天	内置
`googlechat`	Google Chat	Service Account

插件可添加更多渠道（如 Mattermost、Zalo、WeChat 等）。

4.3 渠道登录

```
# Interactive login (all channels)
openclaw channels login

# Channel-specific
openclaw channels login --channel telegram
openclaw channels login --channel discord

# Check status
openclaw channels status --probe
```

4.4 访问控制

每个渠道支持 **私信策略**和**群组策略**：

4.4.1 私信策略（dmPolicy）

- `pairing`（默认）：未知发送者收到配对码，审批后方可通信
- `allowlist`：仅允许名单内的用户
- `open`：任何人都可以私信
- `disabled`：禁用私信

4.4.2 群组策略（groupPolicy）

- `allowlist`（默认）：仅允许名单内的群组
- `open`：所有群组
- `disabled`：禁用群组

4.4.3 配置示例

```
{
  channels: {
    whatsapp: {
      dmPolicy: "pairing",
      allowFrom: ["+15555550123"],
      groups: {
```

```
        "*": { requireMention: true }
      }
    },
    telegram: {
      enabled: true,
      dmPolicy: "allowlist",
      allowFrom: ["123456789"]
    }
  }
}
```

4.5 路由机制

OpenClaw 将回复**路由回消息来源的渠道**，路由是确定性的：

- 模型**不会**选择渠道
- 回复始终返回到入站消息的来源
- 多渠道场景通过 Binding 规则路由到不同 Agent

4.5.1 会话键格式

- 私信：`agent:<agentId>:<mainKey>`
- 群组：`agent:<agentId>:<channel>:group:<groupId>`
- 线程：在基础键后追加 `:thread:<threadId>`

4.6 多账户

支持多账户的渠道（如 WhatsApp）使用 `accountId` 标识每个登录实例。不同 `accountId` 可路由到不同 Agent，实现一台服务器托管多个号码。

Chapter 5

Binding（路由绑定）

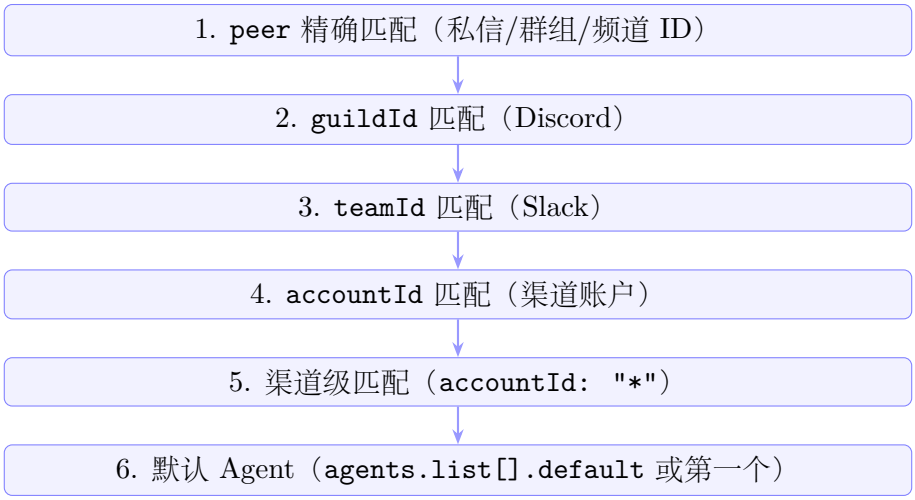
5.1 概述

Binding 是将入站消息路由到指定 Agent 的规则。在多智能体模式下，Binding 决定哪个”大脑”处理哪些消息。

配置路径：`agents.list[].bindings[]`（或顶层 `bindings[]`）

5.2 路由优先级

路由是**确定性的**，**最具体的优先**：



5.3 核心概念

术语	说明
`agentId`	一个”大脑”（工作区 + 认证 + 会话）
`accountId`	渠道账户实例（如 WhatsApp 的 `"personal"` vs `"biz"`）
`binding`	通过 (channel, accountId, peer) 路由到 agentId
`peer`	对端标识（`kind: "dm"|"group"|"channel"` + `id`）

5.4 配置示例

5.4.1 两个 WhatsApp 账户 → 两个 Agent

```
{
  agents: {
    list: [
      { id: "home", default: true, workspace: "~/.openclaw/
   workspace-home" },
      { id: "work", workspace: "~/.openclaw/workspace-work" }
    ]
  },
  bindings: [
    { agentId: "home", match: { channel: "whatsapp", accountId: "
   personal" } },
    { agentId: "work", match: { channel: "whatsapp", accountId: "
   biz" } }
  ]
}
```

5.4.2 一个号码按发送者分流

```
{
  agents: {
    list: [
      { id: "alex", workspace: "~/.openclaw/workspace-alex" },
      { id: "mia", workspace: "~/.openclaw/workspace-mia" }
    ]
  },
  bindings: [
    { agentId: "alex", match: { channel: "whatsapp", peer: { kind:
   "dm", id: "+15551230001" } } },
    { agentId: "mia", match: { channel: "whatsapp", peer: { kind: "
   dm", id: "+15551230002" } } }
  ]
}
```

5.4.3 特定群组路由到特定 Agent

```
{
  bindings: [
    {
      agentId: "work",
      match: {
        channel: "whatsapp",
        accountId: "personal",
        peer: { kind: "group", id: "1203630...@g.us" }
      }
    }
  ]
}
```

5.5 验证

```
openclaw agents list --bindings
```

Chapter 6

Session（会话）

6.1 概述

Session（会话）是用户与智能体之间一次连续对话的容器，包含上下文历史、令牌计数和路由元数据。

配置路径：`session.*`

6.2 会话所有权

所有会话状态都**由 Gateway 网关拥有**。UI 客户端（macOS 应用、WebChat 等）必须向 Gateway 网关查询，而不是读取本地文件。

6.2.1 存储位置

- 存储文件：`~/.openclaw/agents/<id>/sessions/sessions.json`
- 对话记录：`~/.openclaw/agents/<id>/sessions/<sessionId>.jsonl`

6.3 会话键映射

来源	会话键格式
私信（默认）	`agent:<agentId>:<mainKey>`
群组聊天	`agent:<agentId>:<channel>:group:<groupId>`
线程/话题	基础键 + `:thread:<threadId>`
定时任务	`cron:<jobId>`
Webhook	`hook:<hookId>`

6.4 私信分组策略（dmScope）

`session.dmScope` 控制私信如何分组：

值	行为
main（默认）	所有私信共享主会话
per-peer	按发送者 ID 隔离
per-channel-peer	按渠道 + 发送者隔离
per-account-channel-peer	按账户 + 渠道 + 发送者隔离

6.5 会话生命周期

6.5.1 重置策略

- **每日重置**（默认）：Gateway 主机本地时间凌晨 4:00
- **空闲重置**（可选）：滑动空闲窗口（idleMinutes）
- 两者同时配置时，先过期者触发新会话

6.5.2 手动重置

在聊天中发送 /new 或 /reset 创建新会话。

6.6 配置示例

```
{
  session: {
    dmScope: "main",
    reset: {
      mode: "daily",
      atHour: 4,
      idleMinutes: 120
    },
    resetByType: {
      dm: { mode: "idle", idleMinutes: 240 },
      group: { mode: "idle", idleMinutes: 120 }
    },
    identityLinks: {
      alice: ["telegram:123456789", "discord:987654321012345678"]
    }
  }
}
```

`identityLinks` 可将同一用户的不同渠道 ID 映射到统一身份，实现跨渠道共享会话。

6.7 常用命令

```
openclaw status                          # Show sessions
openclaw sessions --json                 # Export all sessions
/status                                  # In chat: check session info
/new                                     # In chat: start new session
/compact                                 # In chat: compact context
/stop                                    # In chat: abort current run
```

Chapter 7

Plugin（插件）

7.1 概述

Plugin（插件）是 OpenClaw 的扩展模块，可以注册：

- **智能体工具**（JSON 模式函数）
- **渠道适配器**（新的聊天平台）
- **生命周期钩子**（拦截消息/工具/会话事件）
- **记忆提供者**

配置路径：`plugins.entries.*`

7.2 安装插件

```
# From npm (recommended)
openclaw plugins install @openclaw/voice-call

# From local folder (dev)
openclaw plugins install ./extensions/my-plugin
```

安装后需**重启 Gateway 网关**加载插件。

7.3 插件清单

每个插件必须提供 `openclaw.plugin.json`：

```
{
  "id": "voice-call",
  "configSchema": {
    "type": "object",
    "additionalProperties": false,
```

```
    "properties": {}
  }
}
```

可选字段：kind、channels、providers、skills、name、description。

7.4 注册智能体工具

```
import { Type } from "@sinclair/typebox";

export default function (api) {
  api.registerTool({
    name: "my_tool",
    description: "Do a thing",
    parameters: Type.Object({
      input: Type.String(),
    }),
    async execute(_id, params) {
      return { content: [{ type: "text", text: params.input }] };
    },
  });
}
```

7.4.1 可选工具

可选工具需显式添加到智能体允许列表：

```
{
  agents: {
    list: [{
      id: "main",
      tools: {
        allow: ["workflow_tool", "group:plugins"]
      }
    }]
  }
}
```

7.5 插件钩子

插件可以注册以下生命周期钩子：

钩子	时机
before_agent_start	运行开始前，注入上下文
agent_end	完成后，检查结果
before_tool_call	工具调用前，拦截参数
after_tool_call	工具调用后，拦截结果
message_received	入站消息
message_sending	出站消息（发送前）
session_start / session_end	会话生命周期
gateway_start / gateway_stop	Gateway 生命周期

7.6 配置示例

```
{
  plugins: {
    entries: {
      "voice-call": {
        enabled: true,
        config: {
          provider: "twilio",
          fromNumber: "+15550001234",
          twilio: {
            accountSid: "ACxxxxxxxx",
            authToken: "..."
          }
        }
      }
    }
  }
}
```

7.7 社区插件

可通过 npm 安装社区维护的插件，例如：

- **WeChat**：`openclaw plugins install @icesword760/openclaw-wechat`
- **Voice Call**：`openclaw plugins install @openclaw/voice-call`
- **Zalo**：`openclaw plugins install @openclaw/zalouser`

Chapter 8

Skill（技能）

8.1 概述

Skill（技能）是教智能体如何使用工具的指令包。每个 Skill 是一个包含 `SKILL.md`（带 YAML frontmatter）的目录，在智能体运行时注入到系统提示中。

配置路径：`skills.entries.*`

8.2 加载位置与优先级

Skills 从三个位置加载，同名时按优先级覆盖：

额外 Skills 文件夹可通过 `skills.load.extraDirs` 配置。

8.3 SKILL.md 格式

```
---
name: nano-banana-pro
description: Generate or edit images via Gemini 3 Pro Image
metadata:
  {"openclaw": {"requires": {"bins": ["uv"], "env": ["
   GEMINI_API_KEY"]}, "primaryEnv": "GEMINI_API_KEY"}}
---

## Instructions

Use the `nano-banana-pro` tool to generate images...
```

8.3.1 Frontmatter 关键字段

字段	说明
`name`	Skill 名称（必填）
`description`	简短描述（必填）
`user-invocable`	是否暴露为斜杠命令（默认 true）
`metadata.openclaw.requires.bins`	所需二进制文件
`metadata.openclaw.requires.env`	所需环境变量
`metadata.openclaw.requires.config`	所需配置项
`metadata.openclaw.primaryEnv`	关联的 API 密钥变量名

8.4 门控（加载时过滤）

OpenClaw 在加载时根据以下条件过滤 Skills：

- `requires.bins`：二进制文件必须存在于 `PATH`
- `requires.env`：环境变量必须存在或在配置中提供
- `requires.config`：配置项必须为真值
- `os`：平台限制（`darwin/linux/win32`）

没有 `metadata.openclaw` 的 Skill 始终加载（除非在配置中禁用）。

8.5 配置覆盖

```
{
  skills: {
    entries: {
      "nano-banana-pro": {
        enabled: true,
        apiKey: "YOUR_API_KEY",
        env: { CUSTOM_VAR: "value" }
      },
      "dangerous-skill": { enabled: false }
    }
  }
}
```

8.6 ClawHub（Skills 注册表）

```
clawhub install <skill-name>    # Install to workspace
clawhub update --all            # Update all skills
clawhub sync --all              # Scan + publish updates
```

浏览 Skills：https://clawhub.com

8.7 安全注意事项

- 将第三方 Skills 视为**不受信任的代码**，启用前请阅读
- 高风险工具优先使用沙箱隔离运行
- `skills.entries.*.apiKey` 将秘密注入到宿主机进程（非沙箱）

Chapter 9

Memory（记忆）

9.1 概述

OpenClaw 记忆是**智能体工作空间中的纯 Markdown 文件**。模型只”记住”写入磁盘的内容——记忆文件是唯一的事实来源。

配置路径：`memorySearch.*`

9.2 记忆文件

默认工作空间布局使用两个记忆层：

文件	用途	加载时机
`memory/YYYY-MM-DD.md`	每日日志（仅追加）	会话开始时读取今天和昨天
`MEMORY.md`	精心整理的长期记忆	仅在主要私人会话中加载

位于工作空间下（默认 `~/.openclaw/workspace`）。

9.3 何时写入记忆

- 决策、偏好和持久性事实 → `MEMORY.md`
- 日常笔记和运行上下文 → `memory/YYYY-MM-DD.md`
- 用户说”记住这个”时 → 立即写入

提示：如果你想让某些内容持久保存，请**明确要求**智能体将其写入记忆。

9.4 自动记忆刷新

当会话接近自动压缩时，OpenClaw 触发一个**静默智能体回合**，提醒模型在上下文压缩前写入持久记忆。

```
{
  agents: {
    defaults: {
      compaction: {
        memoryFlush: {
          enabled: true,
          softThresholdTokens: 4000
        }
      }
    }
  }
}
```

9.5 向量记忆搜索

OpenClaw 在记忆文件上构建小型向量索引，支持语义查询（即使措辞不同也能找到相关笔记）。

9.5.1 默认行为

- 默认启用，监视记忆文件变更
- 自动选择嵌入提供商：本地 → OpenAI → Gemini
- 使用 sqlite-vec 加速向量搜索

9.5.2 配置提供商

```
{
  agents: {
    defaults: {
      memorySearch: {
        provider: "openai",
        remote: { apiKey: "YOUR_OPENAI_API_KEY" }
      }
    }
  }
}
```

支持的提供商：`openai`、`gemini`、`local`（node-llama-cpp）。

9.5.3 额外记忆路径

索引默认工作空间之外的 Markdown 文件：

```
{
  agents: {
    defaults: {
      memorySearch: {
        extraPaths: ["../team-docs", "/srv/shared-notes/overview.md
  "]
      }
    }
  }
}
```

9.6 全文搜索

除向量搜索外，OpenClaw 还支持基于 SQLite FTS5 的全文搜索，可与向量搜索结合使用以提高召回率。

9.7 记忆搜索工具

记忆搜索由活动的记忆插件提供（默认：`memory-core`）。禁用方式：

```
{ plugins: { slots: { memory: "none" } } }
```

Chapter 10

Sandbox（沙箱隔离）

10.1 概述

OpenClaw 可以**在 Docker 容器内运行工具**以减少影响范围。沙箱隔离是**可选的**，由配置控制。Gateway 网关本身保留在主机上，仅工具执行在容器中隔离运行。

配置路径：`agents.defaults.sandbox.*`

10.2 什么会被沙箱隔离

隔离	不隔离
工具执行（exec, read, write, edit 等）	Gateway 网关进程本身
沙箱浏览器（可选）	提权工具（`tools.elevated`）
	明确允许在主机运行的工具

10.3 沙箱模式

`agents.defaults.sandbox.mode`：

模式	行为
`off`	不使用沙箱
`non-main`	仅非主会话使用沙箱（默认）
`all`	所有会话都在沙箱中运行

10.4 作用域

`agents.defaults.sandbox.scope`：

- `session`（默认）：每个会话一个容器
- `agent`：每个智能体一个容器
- `shared`：所有沙箱会话共享一个容器

10.5 工作区访问

`agents.defaults.sandbox.workspaceAccess:`

值	行为
none（默认）	使用沙箱工作区（~/.openclaw/sandboxes）
ro	只读挂载智能体工作区
rw	读写挂载智能体工作区

10.6 最小启用示例

```
{
  agents: {
    defaults: {
      sandbox: {
        mode: "all",
        scope: "session",
        workspaceAccess: "none",
        docker: {
          image: "openclaw-sandbox:bookworm-slim"
        }
      }
    }
  }
}
```

10.7 镜像构建

```
# Build sandbox image
scripts/sandbox-setup.sh

# Build sandbox browser image (optional)
scripts/sandbox-browser-setup.sh
```

默认沙箱容器**没有网络**，通过 `docker.network` 可覆盖。

10.8 自定义绑定挂载

将额外主机目录挂载到容器：

```
{
  agents: {
    defaults: {
      sandbox: {
        docker: {
          binds: [
            "/home/user/source:/source:ro",
            "/var/run/docker.sock:/var/run/docker.sock"
          ]
        }
      }
    }
  }
}
```

10.9 setupCommand

`setupCommand` 在容器创建后**运行一次**（非每次运行），用于安装额外依赖：

```
{
  agents: {
    defaults: {
      sandbox: {
        docker: {
          network: "bridge",
          setupCommand: "apt-get update && apt-get install -y
   python3"
        }
      }
    }
  }
}
```

注意：包安装需要网络出口（`network: "bridge"`）和可写根文件系统。

10.10 多智能体覆盖

每个智能体可独立覆盖沙箱配置:

```
{
  agents: {
    list: [{
      id: "build",
      sandbox: {
        mode: "all",
        docker: {
          binds: ["/mnt/cache:/cache:rw"]
        }
      }
    }]
  }
}
```

10.11 调试

```
openclaw sandbox explain  # Check effective sandbox config
```

Chapter 11

实战：服务器部署与配置详解

基于实际操作记录，涵盖远程服务器部署 OpenClaw、API 配置、目录结构解析、Agent 与会话管理等内容。

11.1 远程服务器部署

11.1.1 环境信息

项目	值
服务器 IP	`<your-server-ip>`
用户名	ubuntu
操作系统	Ubuntu Linux 6.8.0-51-generic (x86_64)
私钥文件	`~/.ssh/your-key.pem`

11.1.2 SSH 连接

```
# 私钥权限必须为 600，否则 SSH 会拒绝连接
chmod 600 ~/.ssh/your-key.pem

# 连接服务器
ssh -i ~/.ssh/your-key.pem ubuntu@<your-server-ip>
```

注意：如果私钥权限过宽（如 644），SSH 会报错 `Permissions 0644 for key are too open`，必须先修复权限。

11.1.3 安装 OpenClaw

系统要求

- **Node.js 22+**（安装器脚本会自动检测并安装）
- macOS、Linux 或通过 WSL2 的 Windows

- pnpm 仅在从源代码构建时需要

方式一：安装器脚本（推荐）

安装器脚本会自动处理 Node.js 检测、安装以及新手引导，一步到位：

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

安装器会自动执行以下步骤：

1. 检测操作系统（macOS / Linux / WSL）
2. 确保 Node.js 22+ 已安装（Linux 通过 NodeSource 安装）
3. 确保 Git 已安装
4. 通过 npm 全局安装 OpenClaw
5. 运行新手引导（onboarding）

如需跳过新手引导（适用于自动化部署场景）：

```
curl -fsSL https://openclaw.ai/install.sh | bash -s -- --no-onboard
```

方式二：全局 npm 安装（手动）

如果服务器上已经有 Node.js 22+，可以直接通过 npm 安装：

```
# 确认 Node.js 版本 >= 22
node -v

# 全局安装 OpenClaw
npm install -g openclaw@latest
```

方式三：从源代码构建

适用于需要自定义修改或参与开发的场景：

```
git clone https://github.com/openclaw/openclaw.git
cd openclaw
pnpm install
pnpm build

# 将 openclaw 命令链接到全局
pnpm link --global
```

安装后验证

```
# 检查安装是否成功
openclaw --version

# 运行诊断检查
openclaw doctor

# 如果跳过了新手引导，手动执行
openclaw onboard --install-daemon
```

提示：如果安装完成后在新终端中找不到 `openclaw` 命令，通常是 Node/npm 的 PATH 配置问题。可以将 npm 全局 bin 目录加入 PATH：

```
export PATH="$(npm prefix -g)/bin:$PATH"
```

建议将上述命令添加到 `~/.bashrc` 或 `~/.zshrc` 中以持久生效。

安装完成后，通过 `openclaw` 命令即可使用。

11.2 API 配置

11.2.1 配置方式

推荐使用 `openclaw config set` 命令逐项配置，避免手动编辑 JSON 格式出错。

```
# 设置 API Provider（以某个 Provider 为例）
openclaw config set secrets.providers.<provider>.apiKey "your-api-
    key"
openclaw config set secrets.providers.<provider>.apiUrl "https://<
    api-endpoint>/v1/"

# 设置模型
openclaw config set models.<provider>/Kimi-K2.5.provider "<provider
    >"
openclaw config set models.<provider>/Kimi-K2.5.api "openai-
    completions"

# 设置默认模型
openclaw config set defaults.model "<provider>/Kimi-K2.5"
```

```
# 设置网关模式为本地
openclaw config set gateway.mode "local"
```

11.2.2 配置文件结构

配置文件位于 ~/.openclaw/openclaw.json，核心结构如下：

```
{
  "secrets": {
    "providers": {
      "<provider>": {
        "apiKey": "your-api-key",
        "apiUrl": "https://<api-endpoint>/v1/"
      }
    }
  },
  "models": {
    "<provider>/Kimi-K2.5": {
      "provider": "<provider>",
      "api": "openai-completions"
    }
  },
  "defaults": {
    "model": "<provider>/Kimi-K2.5"
  },
  "gateway": {
    "mode": "local"
  }
}
```

11.2.3 关键字段说明

11.2.4 常见配置错误

11.2.5 启动网关

```
# 前台运行（调试用）
openclaw gateway
```

字段	说明
secrets.providers.<name>.apiKey	API 密钥
secrets.providers.<name>.apiUrl	API 端点地址
models.<model>.provider	模型对应的 Provider 名称
models.<model>.api	API 协议类型（如 openai-completions）
defaults.model	默认使用的模型
gateway.mode	网关模式，local 为本地模式

错误	原因	解决方案
config validate 失败	api 类型写成了 openai-chat	改为 openai-com
No API provider registered for api: undefined	模型配置缺少 api 字段	添加 api: "open
网关未启动	未设置 gateway.mode 或未启动	设置为 local，执

```
# 后台运行
nohup openclaw gateway &
```

11.2.6 验证配置

```
# 验证配置是否正确
openclaw config validate

# 测试 Agent 调用
openclaw agent --agent main --session-id test "你好"
```

11.3 ~/.openclaw/ 目录结构详解

以下为服务器上 ~/.openclaw/ 的**实际完整目录树**（基于实际环境采集）。

```
~/.openclaw/
+-- openclaw.json              # 主配置文件（API密钥、模型、网关、
    渠道等）
+-- openclaw.json.bak          # 配置文件自动备份（每次修改自动生
    成，最多保留5份）
+-- openclaw.json.bak.1
+-- openclaw.json.bak.2
+-- openclaw.json.bak.3
+-- openclaw.json.bak.4
```

```
+-- update-check.json          # 版本更新检查记录（记录上次检查时
    间）
|
+-- agents/                    # Agent 注册目录（每个 Agent 一个子
    目录）
|   +-- main/                  # 默认 Agent "main"
|       +-- agent/
|       |   +-- models.json    # 该 Agent 运行时解析后的模型信息快
    照
|       +-- sessions/
|           +-- sessions.json  # 会话索引（记录所有会话的元数据、
    Token用量等）
|           +-- <uuid>.jsonl   # 具体会话的消息历史（JSONL格式，每
    行一条消息）
|
+-- canvas/                    # Canvas 画布功能
|   +-- index.html             # 可视化画布的前端页面
|
+-- completions/               # Shell 自动补全脚本
|   +-- openclaw.bash          # Bash 补全
|   +-- openclaw.zsh           # Zsh 补全
|   +-- openclaw.fish          # Fish 补全
|   +-- openclaw.ps1           # PowerShell 补全
|
+-- cron/                      # 定时任务
|   +-- jobs.json              # 定时任务注册表（cron表达式 + 任务
    定义）
|
+-- devices/                   # 设备管理（手机等Node节点）
|   +-- paired.json            # 已配对的设备列表
|   +-- pending.json           # 待配对的设备列表
|
+-- identity/                  # 身份与认证
|   +-- device.json            # 当前设备标识（设备ID、名称等）
|   +-- device-auth.json       # 设备认证凭据
|
+-- logs/                      # 运行日志
|   +-- config-audit.jsonl     # 配置变更审计日志（每次config修改自
    动记录）
|
```

```
+-- memory/                    # 记忆存储
|   +-- main.sqlite            # 默认Agent的SQLite记忆数据库
|
+-- workspace/                 # 默认工作空间（Agent 人设与知识库）
    +-- .git/                  # Git仓库（workspace自带版本控制）
    +-- .clawhub/              # ClawHub 包管理（类似 node_modules
   的锁文件）
    |   +-- lock.json          #   已安装 Skills 的版本锁定文件
    +-- .openclaw/
    |   +-- workspace-state.json  # 工作空间状态（如bootstrap时间）
    +-- SOUL.md                # 核心人设定义（性格、语气、行为准
   则）
    +-- AGENTS.md              # Agent工作流程定义（启动流程、记忆
   管理规范）
    +-- IDENTITY.md            # Agent身份信息（名字、emoji、风格
   等）
    +-- USER.md                # 用户画像（用户偏好、习惯、备注）
    +-- BOOTSTRAP.md           # 首次启动引导脚本（初始化后可删除）
    +-- HEARTBEAT.md           # 心跳任务定义（周期性检查任务）
    +-- TOOLS.md               # 本地工具备忘（环境特有的SSH、设备
   等信息）
    |
    +-- skills/                # 已安装的工作空间级 Skills（优先级
   最高）
        +-- xiaohongshu-mcp/   #   示例：小红书自动化 Skill
            +-- .clawhub/
            |   +-- origin.json    # 安装来源（registry地址、版本、
   安装时间）
            +-- _meta.json         # Skill 元数据（作者ID、slug、版
   本、发布时间）
            +-- SKILL.md           # Skill 定义文件（名称、描述、使
   用指南）
            +-- scripts/           # Skill 附带的脚本
                +-- xhs_client.py  # 小红书 MCP 客户端 Python 脚本
```

11.3.1 根级文件详解

openclaw.json — 主配置文件

系统的核心配置，包含以下顶级字段：

字段	说明
secrets.providers	API Provider 的密钥和端点
models	可用模型列表及其 Provider 映射、API 协议类型
defaults	默认模型、默认 Agent 等全局缺省值
gateway	网关模式（local/cloud）和端口配置
channels	渠道连接配置（Telegram/飞书/WhatsApp 等）

openclaw.json.bak* — 配置自动备份

每次通过 `openclaw config set` 或其他方式修改配置时，系统会自动备份上一版配置。最多保留 **5 份**备份（`.bak` ~ `.bak.4`），按时间倒序排列，最新的为 `.bak`。

如果配置改坏了，可以直接 `cp openclaw.json.bak openclaw.json` 回退。

update-check.json — 版本检查记录

```
{ "lastCheckedAt": "2026-03-06T14:18:59.388Z" }
```

记录上次检查 OpenClaw 新版本的时间，避免频繁请求更新服务器。

11.3.2 agents/ — Agent 注册与会话存储

每个 Agent 拥有独立的子目录，内含运行时数据：

```
agents/main/
+-- agent/
|   +-- models.json        # 运行时模型快照
+-- sessions/
    +-- sessions.json      # 会话索引
    +-- <uuid>.jsonl       # 会话消息历史
```

agent/models.json — 运行时模型信息

Gateway 启动时从主配置解析生成，包含模型的完整信息：

```
{
  "providers": {
    "<provider>": {
      "baseUrl": "https://<api-endpoint>/v1/",
      "apiKey": "***",
      "models": [{
        "id": "Kimi-K2.5",
```

```
        "api": "openai-completions",
        "reasoning": false,
        "input": ["text"],
        "contextWindow": 200000,
        "maxTokens": 8192,
        "cost": { "input": 0, "output": 0, "cacheRead": 0, "
    cacheWrite": 0 }
      }]
     }
   }
}
```

sessions/sessions.json — 会话索引

记录该 Agent 下所有会话的元数据，键名格式为 `agent:<agent 名 >:< 会话名 >:`

```
{
  "agent:main:main": {
    "sessionId": "<session-uuid>",
    "sessionFile": "~/.openclaw/agents/main/sessions/<session-uuid
   >.jsonl",
    "model": "Kimi-K2.5",
    "modelProvider": "<provider>",
    "contextTokens": 200000,
    "inputTokens": 32,
    "outputTokens": 162,
    "totalTokens": 13050,
    "cacheRead": 13018,
    "skillsSnapshot": { }
  }
}
```

可以从这里看到每个会话的模型、Token 用量、关联的 Skills 等信息。

sessions/<uuid>.jsonl — 会话消息历史

以 JSONL 格式（每行一条 JSON）存储完整的消息交互记录，包括用户消息、Agent 回复、工具调用等。文件名为会话的 UUID。

11.3.3 canvas/ — Canvas 画布

```
canvas/
+-- index.html
```

提供一个本地 Web 可视化画布页面，Agent 可以在上面渲染图表、代码预览等富内容。通过浏览器访问。

11.3.4 completions/ — Shell 自动补全

```
completions/
+-- openclaw.bash
+-- openclaw.zsh
+-- openclaw.fish
+-- openclaw.ps1
```

安装时自动生成的各 Shell 补全脚本，使 `openclaw` 命令支持 Tab 自动补全。加载方式：

```
# Bash
source ~/.openclaw/completions/openclaw.bash

# Zsh
source ~/.openclaw/completions/openclaw.zsh
```

11.3.5 cron/ — 定时任务

```
// cron/jobs.json
{ "version": 1, "jobs": [] }
```

注册 Agent 的定时任务。Agent 可以通过 `cron` Skill 创建周期性任务（如定时发送提醒、定时检查系统状态等），任务定义存储在 `jobs.json` 中。

11.3.6 devices/ — 设备管理

```
devices/
+-- paired.json      # 已配对设备
+-- pending.json     # 待配对设备
```

用于管理通过 WebSocket 连接到 Gateway 的外部设备（手机、平板等 Node 节点）。设备可以暴露硬件能力（摄像头、GPS、通知等），Agent 通过 Gateway 远程调用这些能力。

11.3.7 identity/ — 身份认证

```
identity/
+-- device.json          # 当前设备标识信息
+-- device-auth.json     # 设备认证凭据
```

标识当前安装实例的唯一身份，用于多设备场景下的认证与区分。

11.3.8 logs/ — 审计日志

```
logs/
+-- config-audit.jsonl
```

以 JSONL 格式记录每次配置变更的完整审计信息，包括：

- **时间戳**（ts）
- **触发命令**（argv），如 openclaw config set gateway.mode local
- **变更前后的配置哈希**（previousHash / nextHash）
- **文件大小变化**（previousBytes / nextBytes）
- **可疑操作标记**（suspicious）

  便于排查配置问题和追溯变更历史。

11.3.9 memory/ — 记忆数据库

```
memory/
+-- main.sqlite
```

使用 SQLite 数据库持久化 Agent 的长期记忆。文件名对应 Agent 名称（main.sqlite 对应 main Agent）。存储内容包括：

- 对话摘要
- 用户偏好
- 重要事件
- 提取的关键信息

11.3.10 workspace/ — 工作空间（人设与知识库）

这是 Agent 的”灵魂所在”，包含人设定义、工作流程和已安装的 Skills：

```
workspace/
+-- .git/                          # 自带Git版本控制
+-- .clawhub/                      # ClawHub 包管理
|   +-- lock.json                  # Skills 版本锁定文件
+-- .openclaw/workspace-state.json # 工作空间状态
+-- SOUL.md          # 核心人设
+-- AGENTS.md        # 工作流程
+-- IDENTITY.md      # 身份信息
+-- USER.md          # 用户画像
+-- BOOTSTRAP.md     # 首次引导
+-- HEARTBEAT.md     # 心跳任务
+-- TOOLS.md         # 本地工具备忘
|
+-- skills/          # 通过 ClawHub 安装的工作空间级 Skills
    +-- <skill-name>/
        +-- .clawhub/origin.json   # 安装来源信息
        +-- _meta.json             # Skill 元数据
        +-- SKILL.md               # Skill 定义文件
        +-- scripts/               # Skill 附带脚本（可选）
```

SOUL.md — 核心人设定义

定义 Agent 的性格和行为准则，**是最重要的人设文件**。默认内容要点：

- **真诚帮助**：跳过客套话（如”Great question!”），直接解决问题
- **有观点**：允许有偏好和意见，不做没有个性的搜索引擎
- **先自己查**：先读文件、查上下文、搜索，实在找不到再问用户
- **尊重隐私**：用户给了访问权限，不要滥用，隐私信息不外泄
- **对外谨慎，对内大胆**：对外操作（发邮件、发推）要确认，对内操作（读文件、整理）可以主动

AGENTS.md — Agent 工作流程定义

规定 Agent 每次启动时的标准流程：

1. 读取 `SOUL.md` — 知道自己是谁
2. 读取 `USER.md` — 知道在帮谁
3. 读取 `memory/YYYY-MM-DD.md`（今天 + 昨天）— 获取近期上下文
4. 如果是主会话，还要读取 `MEMORY.md` — 长期记忆

还定义了记忆管理规范：日志写到 `memory/YYYY-MM-DD.md`，精华提炼到 `MEMORY.md`。

IDENTITY.md — Agent 身份信息

存储 Agent 的具体身份标识：名字、物种/形态、沟通风格、代表 emoji 等。在首次引导（Bootstrap）时由用户和 Agent 共同确定。

USER.md — 用户画像

记录用户的基本信息和偏好：

```
- **Name:** （用户姓名）
- **What to call them:** （如何称呼）
- **Timezone:** （时区）
- **Notes:** （备注、喜好、项目等）
```

Agent 通过持续交互不断更新此文件，以便更好地理解和服务用户。

BOOTSTRAP.md — 首次启动引导

新安装后的”出生引导”脚本，引导 Agent 完成：

1. 了解用户（名字、称呼、时区）
2. 确定自己的身份（名字、风格、emoji）
3. 更新 `IDENTITY.md`、`USER.md`、`SOUL.md`
4. 可选配置消息渠道（WhatsApp/Telegram 等）
5. **完成后删除此文件**（不再需要引导）

HEARTBEAT.md — 心跳任务

定义 Agent 需要周期性执行的检查任务。文件为空或仅有注释时，跳过心跳调用。添加任务后，Agent 会按周期自动执行。

TOOLS.md — 本地工具备忘

记录当前环境特有的工具和设备信息，如：SSH 主机别名、摄像头名称、TTS 语音偏好、智能设备昵称等。Skills 是通用的，TOOLS.md 是" 你的环境独有的"。

.clawhub/lock.json — Skills 版本锁定

类似 `package-lock.json`，记录通过 ClawHub 安装的所有 Skills 的版本和安装时间：

```
{
  "version": 1,
  "skills": {
    "xiaohongshu-mcp": {
      "version": "1.0.0",
      "installedAt": 1772864060491
    }
  }
}
```

skills/ — 已安装的工作空间级 Skills

通过 ClawHub 市场安装的 Skills 会存放在此目录下。每个 Skill 是一个独立子目录，内部结构：

```
skills/<skill-name>/
+-- .clawhub/
|   +-- origin.json        # 安装来源（registry、slug、版本、安装时间）
+-- _meta.json             # Skill 元数据（作者ID、slug、版本、发布时间）
+-- SKILL.md               # Skill 定义文件（名称、描述、使用指南）
+-- scripts/               # Skill 附带的可执行脚本（可选）
```

origin.json — 记录 Skill 从哪里安装的：

```
{
  "version": 1,
  "registry": "https://clawhub.ai",
  "slug": "xiaohongshu-mcp",
  "installedVersion": "1.0.0",
  "installedAt": 1772864060489
}
```

_meta.json — Skill 的发布元数据：

```
{
  "ownerId": "<owner-id>",
  "slug": "xiaohongshu-mcp",
  "version": "1.0.0",
  "publishedAt": 1769938539127
}
```

SKILL.md — Skill 的核心定义文件，包含 YAML frontmatter（名称、描述、触发条件）和 Markdown 正文（使用指南、命令说明）。

> **与内置 Skills 的区别**：内置 Skills 在 `/usr/lib/node_modules/openclaw/skills/` 下，不含 `.clawhub/` 和 `_meta.json`；通过 ClawHub 安装的 Skills 带有完整的来源追踪信息。

实例：已安装的 xiaohongshu-mcp Skill

当前服务器上已安装了小红书自动化 Skill，它提供：

功能	命令	说明
检查登录状态	`python scripts/xhs_client.py status`	确认 MCP 服
搜索笔记	`python scripts/xhs_client.py search " 关键词"`	按关键词搜索
获取笔记详情	`python scripts/xhs_client.py detail <feed_id> <xsec_token>`	获取指定笔记
获取推荐流	`python scripts/xhs_client.py feeds`	获取推荐信息
发布笔记	`python scripts/xhs_client.py publish " 标题" " 内容" " 图片 URL"`	发布图文笔记

> 使用前需先在本地启动 `xiaohongshu-mcp` 服务器（监听 `localhost:18060`）并完成扫码登录。

.openclaw/workspace-state.json — 工作空间状态

```
{ "version": 1, "bootstrapSeededAt": "2026-03-06T14:21:02.015Z" }
```

记录工作空间的初始化时间等状态信息。

.git/ — 版本控制

workspace 目录自带 Git 仓库，Agent 对人设文件的修改可以通过 Git 追踪和回滚。

11.4 Skills 系统详解

11.4.1 Skills 三层存放路径

Skills 分三层存放，**优先级从高到低**：

层级	路径	说明
工作空间 Skills	`~/.openclaw/workspace/skills/`	仅当前工作空间 Agent 可用，优先级**最高**
全局共享 Skills	`~/.openclaw/skills/`	所有 Agent 共享的自定义 Skills
内置 Skills	`/usr/lib/node_modules/openclaw/skills/`	随 OpenClaw 安装自带，优先级**最低**

同名 Skill 时，高优先级路径的会覆盖低优先级的。

11.4.2 Skill 的目录结构

每个 Skill 就是一个目录，核心是 `SKILL.md` 文件。根据来源不同，目录结构有所差异：

手动创建 / 内置 Skill（最简结构）：

```
weather/
+-- SKILL.md        # Skill 定义文件
```

通过 ClawHub 市场安装的 Skill（完整结构）：

```
xiaohongshu-mcp/
+-- .clawhub/
|   +-- origin.json        # 安装来源（registry URL、版本、安装时
    间）
+-- _meta.json             # 发布元数据（作者ID、slug、版本、发布时
    间）
+-- SKILL.md               # Skill 定义文件
+-- scripts/               # 附带的可执行脚本（可选）
    +-- xhs_client.py
```

`SKILL.md` 由两部分组成：

- **Frontmatter**（YAML 头部）：声明名称、描述、依赖要求等元数据
- **正文**（Markdown）：详细的使用指南和命令说明

示例（`weather` Skill）：

```
---
name: weather
description: "Get current weather and forecasts via wttr.in or Open
   -Meteo..."
homepage: https://wttr.in/:help
metadata: { "openclaw": { "emoji": "...", "requires": { "bins": ["
   curl"] } } }
---

# Weather Skill
## Commands
...
```

11.4.3 Skill 的依赖检测与激活

OpenClaw 启动时会自动检测每个 Skill 的依赖是否满足，**缺依赖的 Skill 不会被加载**。

依赖类型有四种：

依赖类型	说明	示例
bins	需要系统中安装的命令行工具	curl、gh、rg、clawhub
env	需要设置的环境变量	OPENAI_API_KEY、GEMINI_API_KEY
os	要求特定操作系统	darwin（仅 macOS）
config	需要在配置文件中设置的字段	channels.discord.token

激活方法就是满足对应依赖，例如：

```
# clawhub skill 缺 clawhub 命令 -> 安装即可激活
sudo npm install -g clawhub

# github/gh-issues skill 缺 gh 命令
sudo apt install gh

# session-logs skill 缺 rg (ripgrep) 命令
sudo apt install ripgrep

# notion skill 缺环境变量
export NOTION_API_KEY="your-key"
```

```
# apple-notes skill 要求 os: darwin -> Linux 上无法激活，仅 macOS
  可用
```

11.4.4 Skills 管理命令

```
# 查看所有 Skills 的状态（哪些可用、哪些缺依赖）
openclaw skills check

# 列出所有 Skills
openclaw skills list

# 仅列出已就绪的 Skills
openclaw skills list --eligible

# 查看某个 Skill 的详细信息（含依赖要求和安装提示）
openclaw skills info <name>
# 示例：
openclaw skills info clawhub
# 输出：  clawhub  Missing requirements
#       Requirements:
#         Binaries:  clawhub
#       Install options:
#         -> Install clawhub (npm)
```

11.4.5 安装第三方 Skills

方式一：通过 ClawHub 市场安装（推荐）

ClawHub 是 OpenClaw 的 Skills 市场（https://clawhub.ai），类似 npm 之于 Node.js：

```
# 先安装 clawhub CLI
sudo npm install -g clawhub

# 搜索、安装、同步 Skills
npx clawhub
```

安装后的文件变化：

> 通过 ClawHub 安装的 Skills 位于 `workspace/skills/` 下（工作空间级，优先级最高），并通过 `.clawhub/lock.json` 锁定版本。

```
workspace/
+-- .clawhub/
| +-- lock.json # <- 新增/更新：记录已安装 Skill 版本
+-- skills/
+-- <skill-name>/ # <- 新增：Skill 目录
+-- .clawhub/origin.json # 安装来源追踪
+-- _meta.json # 作者、版本元数据
+-- SKILL.md # Skill 定义
+-- scripts/ # 附带脚本（如有）
```

方式二：通过 plugins 命令安装

```
# 从本地路径安装（支持 .ts/.js/.zip/.tgz）
openclaw plugins install ./my-skill.zip

# 从 npm 安装
openclaw plugins install some-openclaw-plugin

# 以符号链接方式安装（开发调试用）
openclaw plugins install ./my-skill --link

# 锁定精确版本
openclaw plugins install some-plugin --pin
```

插件管理完整命令：

命令	说明
openclaw plugins install <path-or-spec>	安装插件（本地路径/npm 包）
openclaw plugins uninstall	卸载插件
openclaw plugins update	更新已安装的 npm 插件
openclaw plugins list	列出已安装插件
openclaw plugins info	查看插件详情
openclaw plugins enable / disable	启用/禁用插件
openclaw plugins doctor	排查插件加载问题

方式三：手动放置 Skill 目录

直接创建目录并放入 SKILL.md：

```
# 全局共享（所有 Agent 可用）
mkdir -p ~/.openclaw/skills/my-skill
# 编写 SKILL.md ...
```

```
# 工作空间专属（仅当前工作空间 Agent 可用，优先级最高）
mkdir -p ~/.openclaw/workspace/skills/my-skill
# 编写 SKILL.md ...
```

11.4.6 内置 Skills 一览（51 个）

当前服务器上的内置 Skills 及其依赖状态：

已就绪（5 个）：

Skill	功能
`healthcheck`	主机安全检查与加固
`skill-creator`	创建或更新 Skills
`tmux`	远程控制 tmux 会话
`video-frames`	从视频提取帧或片段
`weather`	天气查询与预报

未就绪（46 个，按类别）：

类别	Skills	典型缺失依赖
AI/模型	`gemini`, `openai-image-gen`, `openai-whisper`, `openai-whisper-api`, `oracle`	env: API Key
笔记/文档	`apple-notes`, `bear-notes`, `notion`, `obsidian`, `nano-pdf`, `summarize`	bins / os: darwi
通讯/社交	`discord`, `slack`, `imsg`, `bluebubbles`, `wacli`, `voice-call`	config / os: dar
开发工具	`coding-agent`, `github`, `gh-issues`, `trello`, `clawhub`, `mcporter`	bins: gh, clawhu
多媒体	`canvas`, `camsnap`, `peekaboo`, `songsee`, `spotify-player`, `gifgrep`	bins / os: darwi
智能家居	`openhue`, `sonoscli`, `blucli`	bins
系统/运维	`session-logs`, `model-usage`	bins: rg / os: da
信息获取	`xurl`, `blogwatcher`, `gog`, `goplaces`, `sag`	bins / env
其他	`1password`, `apple-reminders`, `things-mac`, `himalaya`, 等	bins / env / os

提示：运行 `openclaw skills check` 可以实时查看每个 Skill 具体缺什么。`os: darwin` 标记的 Skill 仅限 macOS，在 Linux 服务器上无法激活。

11.5 Agent 与会话层级关系

11.5.1 层级结构

核心概念：

```
OpenClaw 实例
+-- Agent（智能体）
+-- Session 1（会话 1）
+-- Session 2（会话 2）
+-- Session N（会话 N）
```

- **Agent** 是执行单元，每个 Agent 有独立的配置、人设和能力
- **Session** 是对话上下文，每个 Agent 下可以有多个 Session
- 层级关系：Agent → Session，一对多

11.5.2 Agent 管理

```
# 使用默认 Agent（main）
openclaw agent --agent main --message "你好"

# 不指定 --agent 时，默认使用 main Agent
```

11.5.3 Session（会话）管理

开启新会话

方式一：通过 TUI 交互模式（推荐）

```
# 进入默认会话（main）的 TUI 交互模式
openclaw tui

# 开启一个新的命名会话（指定新名称即自动创建）
openclaw tui --session my-new-session

# 带初始消息开启新会话
openclaw tui --session work --message "帮我整理一下今天的任务"
```

`--session <key>` 是创建新会话的关键参数。不指定时默认使用 `main` 会话，指定一个**新名称**就会自动创建新会话，彼此上下文完全隔离。

方式二：通过 agent 命令（单次对话，非交互）

```
# 在默认会话中发一条消息（复用已有 main 会话上下文）
openclaw agent --agent main --message "你好"

# 使用指定 session-id 复用某个会话
```

```
openclaw agent --agent main --session-id <uuid> --message "继续上次的话题"
```

注意：agent --session-id 是用来**复用已有会话 UUID** 的，不是创建新命名会话。要创建全新会话，推荐使用 openclaw tui --session < 新名称 >。

查看会话列表

```
# 列出所有会话
openclaw sessions

# 仅查看最近 N 分钟活跃的会话
openclaw sessions --active 120

# JSON 格式输出
openclaw sessions --json

# 查看指定 Agent 的会话
openclaw sessions --agent work

# 查看所有 Agent 的会话
openclaw sessions --all-agents
```

清理会话

```
# 预览要清理的会话（不实际执行）
openclaw sessions cleanup --dry-run

# 真正清理过期会话
openclaw sessions cleanup --enforce

# 手动清空所有会话（完全重置）
echo '{}' > ~/.openclaw/agents/main/sessions/sessions.json
```

TUI 完整参数

11.5.4 Agent 独立人设配置

每个 Agent **可以**拥有独立的人设，通过 --workspace 参数指定独立工作空间：

参数	说明
`--session <key>`	会话名称，指定新名称自动创建（默认 main）
`--message <text>`	连接后自动发送的初始消息
`--thinking <level>`	思考等级覆盖
`--deliver`	将回复投递到渠道（默认 false）
`--history-limit <n>`	加载的历史条目数（默认 200）
`--url <url>`	Gateway WebSocket URL
`--token <token>`	Gateway token

```
# 为不同 Agent 指定不同的工作空间
openclaw agent --agent work --workspace ~/openclaw-work --message "
    处理工作邮件"
openclaw agent --agent life --workspace ~/openclaw-life --message "
    今天天气怎么样"
```

不同工作空间下可以有各自独立的 `SOUL.md`，从而实现不同 Agent 拥有不同的人设定义。

> **说明**：默认情况下 `~/.openclaw/workspace/` 是共享的，所有 Agent 使用同一套人设。如果需要独立人设，需通过 `--workspace` 参数为不同 Agent 指定不同的工作空间目录。

11.6 模型切换实战

11.6.1 切换步骤

以从 <Provider>/Kimi-K2.5 切换到 DeepSeek-V3.2 为例：

第一步：备份当前配置

```
cp ~/.openclaw/openclaw.json ~/.openclaw/openclaw.json.bak.old-
    provider
```

第二步：修改配置文件

可通过 Python 脚本修改（避免手动编辑 JSON 出错）：

```
import json, os

config_path = os.path.expanduser('~/.openclaw/openclaw.json')
c = json.load(open(config_path))
```

```
# 替换 models.providers
c['models'] = {
    'providers': {
        'deepseek': {
            'baseUrl': 'https://api.deepseek.com/v1/',
            'apiKey': 'sk-xxxxxxxxxxxxxxxx',
            'models': [{
                'id': 'deepseek-chat',          # <- API 实际模型
   ID
                'name': 'DeepSeek-V3.2',        # <- 显示名称（自定
   义）
                'api': 'openai-completions'
            }]
        }
    }
}

# 更新默认模型引用
c['agents']['defaults']['model'] = {'primary': 'deepseek/deepseek-
   chat'}
c['agents']['defaults']['models'] = {'deepseek/deepseek-chat': {}}

json.dump(c, open(config_path, 'w'), indent=2)
```

第三步：清理运行时缓存和旧会话

这一步至关重要！不清理会导致旧模型信息残留。

```
# 清理运行时模型快照（Gateway 重启后会重新生成）
rm ~/.openclaw/agents/main/agent/models.json

# 清空旧会话（旧会话上下文中包含旧模型信息，会导致新模型"自以为是"
   旧模型）
echo '{}' > ~/.openclaw/agents/main/sessions/sessions.json
```

第四步：重启 Gateway

```
# 杀掉旧进程
pkill -f openclaw-gateway

# 重新启动
```

```
nohup openclaw gateway run --force > /dev/null 2>&1 &

# 验证
openclaw gateway health
openclaw status
```

第五步：测试新模型

```
openclaw agent --agent main --message "你是什么模型?" --json
```

11.6.2 关键注意事项

模型 ID 必须用 API 实际支持的名称

不同 API 提供商的模型 ID 可能与官方宣传名称不同：

```
# 查询 API 实际支持的模型列表
curl -s https://api.deepseek.com/v1/models \
  -H "Authorization: Bearer sk-xxx" | python3 -c \
  "import sys,json; [print(m['id']) for m in json.load(sys.stdin).
   get('data',[])]"
```

DeepSeek 实际模型 ID：

官方名称	API 模型 ID	说明
DeepSeek-V3.2	`deepseek-chat`	通用对话模型
DeepSeek-R1	`deepseek-reasoner`	推理模型

填错模型 ID 会得到 `400 Model Not Exist` 错误。

配置文件中的 id 与 name 的区别

```
{
  "id": "deepseek-chat",       // <- 实际发送给 API 的模型 ID（必须
    准确）
  "name": "DeepSeek-V3.2",     // <- 仅用于显示的友好名称（可自定
    义）
  "api": "openai-completions"
}
```

清理项	路径	原因
运行时模型快照	`agents/main/agent/models.json`	Gateway 重启后会合并新旧配置，导致旧 prov
旧会话	`agents/main/sessions/sessions.json`	旧会话上下文中包含”我是 XX 模型”的历史消
Gateway 进程	`pkill -f openclaw-gateway`	内存中的配置需要重启才能刷新

切换模型后必须清理的三个地方

不清理这三处，即使配置改了，Agent 还是会根据旧的上下文信息回答说自己是旧模型。

11.6.3 快速回退

```
# 切回之前的模型配置
cp ~/.openclaw/openclaw.json.bak.old-provider ~/.openclaw/openclaw.
   json
rm ~/.openclaw/agents/main/agent/models.json
echo '{}' > ~/.openclaw/agents/main/sessions/sessions.json
pkill -f openclaw-gateway
nohup openclaw gateway run --force > /dev/null 2>&1 &
```

11.7 实战踩坑总结

问题	现象	解决
SSH 私钥权限	`Permissions too open` 拒绝连接	`chmod 600 ~/.ssh/your-key`
JSON 配置格式	手动编辑容易出错	用 `openclaw config set` 命令
API 类型错误	`openai-chat` 不被支持	改为 `openai-completions`
模型缺少 api 字段	`No API provider for api: undefined`	配置中添加 `api` 字段
网关未启动	Agent 无法调用	`nohup openclaw gateway &`
模型 ID 写错	`400 Model Not Exist`	用 `/v1/models` 接口查询实际
切换模型后 Agent 仍自称旧模型	配置已改但回复说”我是 Kimi”	清理三处：`models.json` + se
`--session-id` 未创建新会话	期望新建会话但复用了旧的	用 `openclaw tui --session`
Gateway SIGHUP 导致崩溃	发 HUP 信号后 Gateway 挂掉	不要用 `kill -HUP`，直接 pki